

Python 03

Elias Wachmann

2025

Content

1. Variables
2. Python files
3. Functions
4. Fitting Data

Variables

Variable Names

From PEP-8 Style Guide:

- must start with letter or _
- can only contain alpha-numeric values (a-z, A-Z, 0-9) and _
- variable names are case sensitive
- do not overwrite built-in functions

Naming conventions

According to PEP-8 Style Guide:

- `CONST_NAMES` are all caps with `_` between words
- `function_names` are all lower case with `_` between words
- variables follow the same rules as functions
- `ClassNames` are `CamelCase`

Python files

Python files and usage

- Python files are called `.py`
- Python files can be run from the command line
 - `python3 file.py`
- Python files can be imported into other Python files
 - `import numpy`
 - or with an alias `import numpy as np`
- Python files can be run as scripts or imported as modules

Python files run as scripts

I can simply run this file and it will print the sinus values from y for the indices 5 through 10 (excluding 10).

```
1 # myscript.py
2 import numpy as np
3
4 x = np.arange(0, 10, 0.1)
5 y = np.sin(x)
6 print(y[5:10])
```

What happens if I import this file in another file?

Python files imported as modules

I can now import `myscript.py` in another file using the `import` statement:

```
1 import myscript
2 print("This is import.py")
```

What happens if I run this file?

Python files imported as modules

```
• lides] % /bin/python3 /home/etschgi1/Desktop/REPOS/exercises-python/slide  
s24/03/examples/import.py  
[0.47942554 0.56464247 0.64421769 0.71735609 0.78332691]  
This is import.py
```

It not only prints the *This is import.py* line, but also the sinus values from `y` for the indices 5 through 10 (excluding 10).

These are printed because the `import` statement runs the file as a script.

How to avoid this?

```
__name__ == '__main__'
```

By using the `__name__` variable, we can avoid running the file as a script when it is imported as a module.

```
1 # myscript2.py
2 import numpy as np
3
4 if __name__ == "__main__":
5     x = np.arange(0, 10, 0.1)
6     y = np.sin(x)
7     print(y[5:10])
```

Importing this file in another file will not print the sinus values.

Functions

Functions

- Functions are defined with `def`
- Functions can have arguments
- Functions can return values
- Functions can have default arguments
- Functions can have variable number of arguments

Some examples

```
1 def hello():  
2     print("Hello, world!")  
3  
4  
5 def arguments(a, b, c):  
6     print("Got arguments:", a, b, c)  
7  
8  
9 def return_value():  
10    return 42
```

Some examples

```
1 def add(a, b=2):  
2     return a + b  
3  
4  
5 add_two_arguments = add(30, 12)  
6 add_one_argument = add(40)
```

Multi returns

Multiple returns from the same function

```
1 def multireturn(a, b):  
2     sum_ = a + b  
3     product = a * b  
4     return sum_, product  
5  
6  
7 my_sum, my_product = multireturn(3, 4)  
8 print(my_sum)    # 7  
9 print(my_product) # 12
```


Import functions from other files

```
1 import random
2
3
4 def random_add(a, b):
5     return a + b + random.random()
```

Import the functions from myfunc.py in another file (just like we did with numpy):

```
1 from myfunc import random_add
2
3 print(random_add(1, 2))
```

Fitting Data

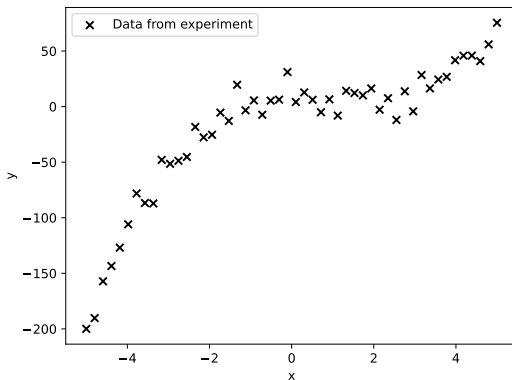
Fitting Data

In physics we often have to fit data to a function.
There are many ways to fit a function to given data.
For now we will use numpy's polyfit() function.

It fits a polynomial of degree `deg` to the data reducing the sum of squared residuals.
It returns the coefficients of the polynomial in decreasing powers.

Fitting Data - Experimental Data

From our measurements we get the following data:



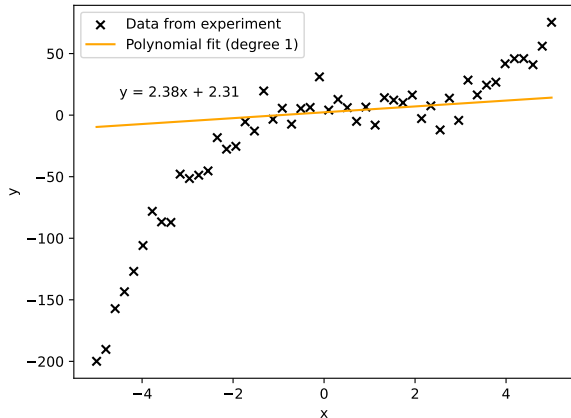
Fitting Data - Linear Fit

```
1 x_data = [...]  
2 y_data = [...]  
3 # plot data  
4 plt.scatter(x_data, y_data, label="Data")  
5 # Fit a line to the data (deg=1)  
6 coefficients1 = np.polyfit(x_data, y_data,  
    1)  
7 # calculate the fitted values  
8 y_fit1 = np.polyval(coefficients1, x_data)  
9 plt.plot(x_data, y_fit1, label='Fit deg 1')
```

polyval() evaluates the polynomial at the given points.

Fitting - Linear Fit

A linear fit on the data works well for $x \in [-2, 3]$, but not outside this range.



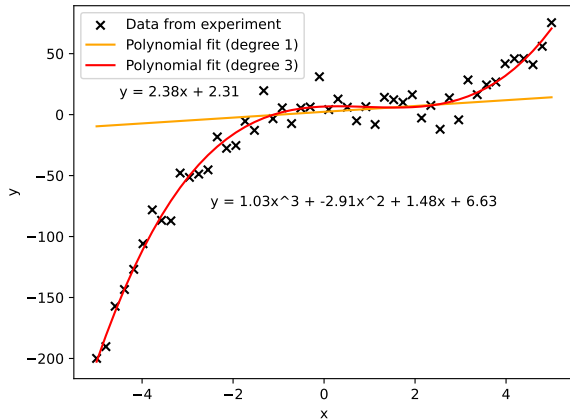
Fitting Data - Cubic Fit

We can now add another fit, this time a cubic fit:

```
1 ...  
2 # Fit a polynomial of degree 3 to the data  
3 coefficients3 = np.polyfit(x_data, y_data,  
4                             3)  
4 y_fit3 = np.polyval(coefficients3, x_data)  
5 plt.plot(x_data, y_fit3, label='Fit deg 3')
```

Fitting - Cubic Fit

A cubic fit describes the data better than a linear fit.



Fitting Data - Uncertainty

Every measurement has it's own uncertainty.
We add uncertainty-bars to the data points using the errorbar() function:

```
1 plt.errorbar(x, y, yerr=y_err, xerr=x_err,
   fmt='o',
2             color='blue', ecolor='red',
   capsize=5)
```

First two arguments are the data, `yerr` and `xerr` are the uncertainties. `color` specifies the color of the datapoints and `ecolor` the color of the errorbars.

Fitting Data - Uncertainty

